Buổi 6: Vòng đời của Activity

Nội dung chính

6.1. Các thành phần cơ bản của ứng dụng Android:

- Activity: Giao diện người dùng, mỗi Activity đại diện cho một màn hình.
- Service: Thực hiện các tác vụ chạy nền.
- Broadcast Receiver: Lắng nghe và xử lý các sự kiện hệ thống.
- Content Provider: Chia sẻ dữ liệu giữa các ứng dụng.

6.2. Vòng đời của Activity:

- Các trạng thái: onCreate(), onStart(), onResume(), onPause(), onStop(), và onDestroy().
- Quản lý tài nguyên và trạng thái trong từng giai đoạn.

6.3. Điều hướng trong ứng dụng:

- Intent và IntentFilter: Điều hướng và trao đổi dữ liệu giữa các thành phần.
- Navigation Component: Điều hướng giữa các màn hình sử dụng Navigation Graph.
- Các phương pháp điều hướng khác: FragmentManager, ViewPager.

6.4. Tạo ứng dụng đa màn hình:

- Thiết kế giao diện đa màn hình sử dụng Activity và Fragment.
- Quản lý trạng thái và dữ liệu giữa các màn hình bằng ViewModel hoặc onSaveInstanceState().

Mục tiêu cần đạt được

- Hiểu các thành phần chính trong ứng dụng Android và vai trò của từng thành phần.
- Nắm rõ vòng đời của Activity và cách quản lý trạng thái trong từng giai đoạn.
- Biết cách sử dụng Intent và IntentFilter để điều hướng giữa các thành phần của ứng dụng.
- Vận dụng Navigation Component và các phương pháp điều hướng khác để xây dựng ứng dụng đa màn hình.
- Quản lý hiệu quả trạng thái và dữ liệu giữa các màn hình trong ứng dụng Android.

6.1. Các thành phần của ứng dụng Android

Một ứng dụng android có 4 thành phần cơ bản như sau:

Hình 6.1: Các thành phần cơ bản của Android

- **Activity:**

Activity là thành phần cốt lõi trong ứng dụng Android, đại diện cho một màn hình giao diện mà người dùng tương tác. Mỗi Activity tương ứng với một màn hình. Chúng ta có thể chuyển đổi giữa các Activity và chia nhỏ giao diện thành nhiều Activity khác nhau để quản lý tốt hơn.

Ví dụ: Màn hình đăng nhập hoặc màn hình hiển thị danh sách sản phẩm.

- **Service:**

Service là thành phần chạy ngầm, không cung cấp giao diện người dùng. Nó xử lý các tác vụ dài hạn hoặc các tác vụ yêu cầu chạy ngầm như phát nhạc, xử lý tệp, hoặc tải dữ liệu từ Internet.

Ví dụ: Một ứng dụng nghe nhạc có thể sử dụng Service để phát nhạc ngay cả khi người dùng rời khỏi Activity chính.

- **Broadcast Receiver:**

Broadcast Receiver là thành phần dùng để nhận và xử lý các sự kiện hệ thống hoặc ứng dụng. Nó cho phép ứng dụng phản hồi lại những sự kiện như kết nối mạng thay đổi, pin yếu, hoặc các thông báo từ ứng dụng khác.

Ví dụ: Ứng dụng nhận thông báo khi thiết bị sạc pin hoặc khi nhận tin nhắn SMS mới.

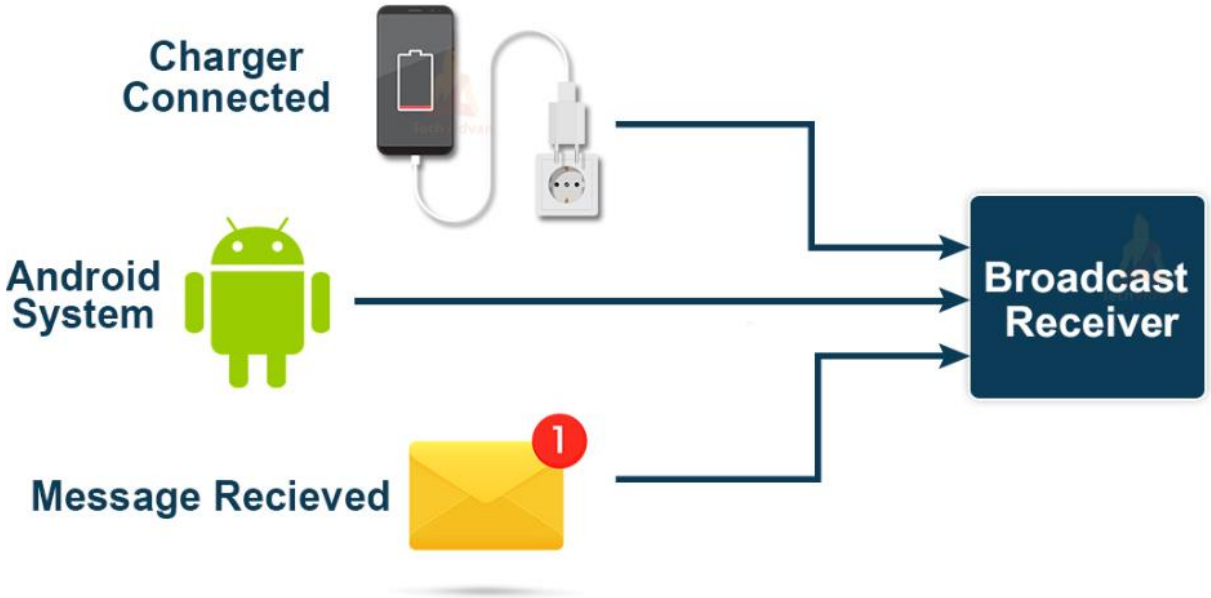
- **Content Provider:**

Content Provider cung cấp một cách để chia sẻ dữ liệu giữa các ứng dụng. Nó cho phép truy cập và lưu trữ dữ liệu từ các ứng dụng khác hoặc từ hệ thống một cách an toàn.

Ví dụ: Ứng dụng truy cập danh bạ của người dùng qua Content Provider.

6.1.1. Đối tượng quản bá thông điệp Broadcast Reciever

Đối tượng Broadcast Reciever cho phép một Activity nhận các thông điệp quảng bá từ các Activity khác. Chúng thường được sử dụng khi muốn nhắc người dùng cần chú ý đến một sự kiện nào nó đã xảy ra trong hệ thống.



Hình 6.2: Broadcast Reciever

Để sử dụng đối tượng Broadcast Reciever, người lập trình cần đăng ký sử dụng nó trong file manifest theo cú pháp như sau:

```
<receiver android:name="ServiceManager">
<intent-filter>
<action android:name="android.intent.action.BOOT_COMPL/>
</intent-filter>
</receiver>
```

Trong đó, thuộc tính android:name chỉ định tên lớp nhận thông điệp quảng bá và thẻ action xác định tên sự kiện xảy ra trong hệ thống hoặc tên lớp thực hiện quảng bá. Ví dụ trong trường hợp này là lớp SeviceManager sẽ nhận thông điệp quảng bá khi quá trình khởi động hệ thống hoàn thành.

Bảng 6.1 dưới đây liệt kê một số sự kiện hệ thống như sau:

Tên sự kiện	Mô tả
Intent.ACTION_BOOT_COMPLETED	Khởi động hệ thống hoàn thành. Yêu cầu cấp quyền truy cập đến thiết bị android.permission.RECEIVE_BOOT _ COM- PLETED
Intent.ACTION_POWER_CONNECTED	Khi sạc nguồn được kết nối

Intent.ACTION_BATTERY_LOW	Cảnh báo khi pin ở mức thấp
Intent.ACTION_BATTERY_OKAY	Trạng thái pin tốt trở lại

Để tạo một thông điệp broadcast từ một Activity người lập trình cần sử dụng cú pháp như sau:

```
val intent = Intent()
// Tên lớp quảng bá thông điệp
intent.action = "com.example.Broadcast"
// Gửi dữ liệu qua Intent
intent.putExtra("HighScore", 1000)
// Quảng bá Intent
sendBroadcast(intent)
```

Để nhận một thông điệp BroadcastReceive người lập trình phải xây dựng lớp con kế thừa lớp BroadcastReceive và viết đè phương thức onReceive() theo cú pháp sau:

```
package ...
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent

class MyReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        // Các lệnh cần thực hiện khi nhận được thông điệp quảng bá
    }
}
```

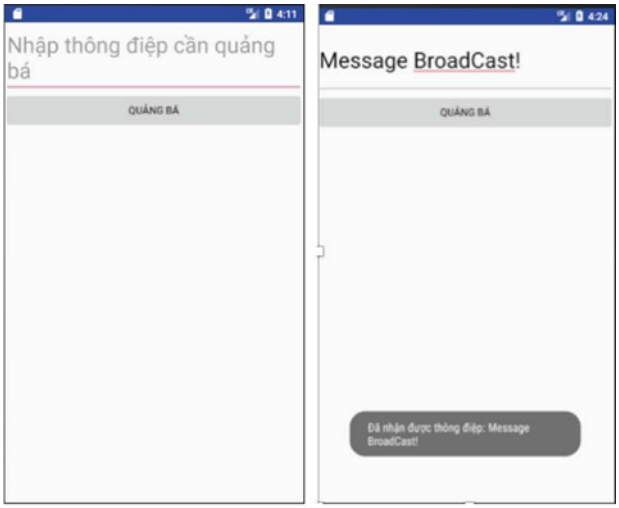
Ngoài ra, người lập trình có thể thực hiện đăng ký nhận thông điệp quảng bá sử dụng cú pháp sau:

```
val filter = IntentFilter("com.example.Broadcast")
val receiver = MyReceiver()
registerReceiver(receiver, filter)
```

Nếu muốn huỷ đăng ký nhận thông điệp quảng bá sử dụng cú pháp lệnh:

```
unregisterReceiver(receiver)
```

Ví dụ: Tạo hai ứng dụng, một ứng dụng gồm một Activity gửi có tên MyActivity gửi một thông điệp quảng bá là “Đây là thông điệp quảng bá”, ứng dụng thứ hai chứa một Activity có tên MyBroadcastReceive đăng ký nhận thông điệp quảng bá từ ứng dụng thứ nhất và hiển thị thông điệp nhận được lên màn hình qua thông điệp Toast.



a. Trước khi quảng bá b. Trước khi quảng bá

Hình 6.3: Kết quả

File activity_main.xml chứa giao diện như sau:

```
import android.app.Activity
import android.content.Intent
import android.os.Bundle
import android.view.View
import android.widget.EditText
class MainActivity : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }

    fun broadcastIntent(view: View) {
        val intent = Intent(this, MyBroadcastReceiver::class.java)
        val et = findViewById<EditText>(R.id.extraIntent)
        intent.putExtra("message", et.text.toString() as CharSequence)
        intent.action = "MainActivity"
        sendBroadcast(intent)
    }
}
```

File MyBroadcastReceiver.java chứa mã như sau:

```
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.widget.Toast

class MyBroadcastReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        val intentData = intent.getCharSequenceExtra("message")
        Toast.makeText(context, "Đã nhận được thông điệp: $intentData",
Toast.LENGTH_LONG).show()
```

```
        }
    }
```

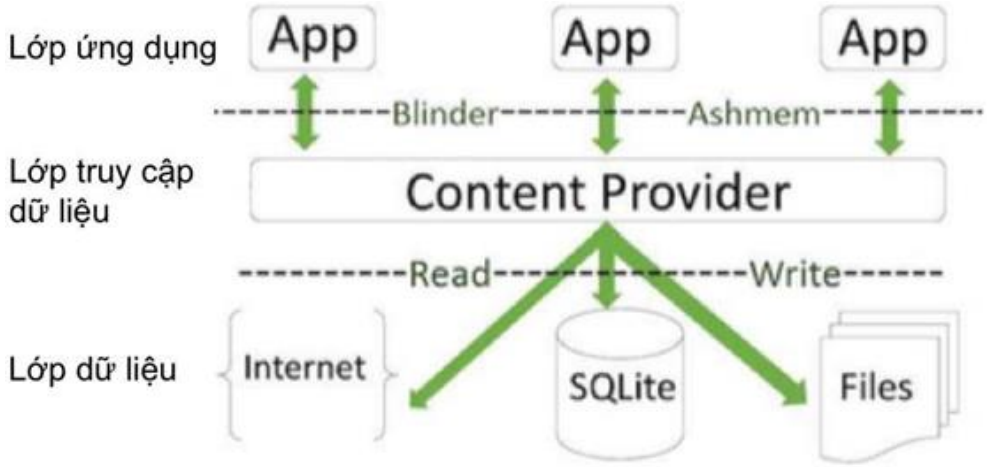
6.1.2. Đối tượng chia sẻ dữ liệu Content provider

Đối tượng Content Provider cho phép một ứng dụng chia sẻ dữ liệu của nó với các ứng dụng khác trong hệ thống. Chẳng hạn như danh bạ (contact) trên thiết bị được sử dụng bởi nhiều ứng dụng như tin nhắn, cuộc gọi, email,..

Các dạng dữ liệu được dùng chung bởi nhiều ứng dụng thường bao gồm: các cặp giá trị dùng chung (preferences), các file dữ liệu được lưu trữ trong bộ nhớ, các cơ sở dữ liệu SQLite trong hệ thống và các dữ liệu chia sẻ qua mạng.

Khai báo lớp ContentProvider như sau:

class MyApplication extends ContentProvider { }



Hình 6.3: ContentProvider

Để truy vấn một Content Provider, người lập trình phải xác định các thành phần trong chuỗi URI theo định dạng:

<prefix>://<authority>/<data_type>/<id>

Các thành phần trong truy chuỗi URI được mô tả như bảng 6.2.

Bảng 6.2 chuỗi URI

Phần	Mô tả
<prefix>	Luôn được thiết lập là content://
<authority>	Chỉ định tên cụ thể của Content Provider (Ví dụ: contacts, browser,. . .). Riêng đối với các các ứng dụng không phải của hệ thống thì bạn phải chỉ định tên đầy đủ tới package của ứng dụng, Ví dụ: com.ictu.androidebook)
<data_type>	Chỉ rõ kiểu dữ liệu (Ví dụ: Để lấy tất cả các liên hệ trong Contacts Content Provider thì kiểu dữ liệu là people và URI sẽ là: con- tent://contacts/people

<id>	Chỉ định rõ một record (Ví dụ: Nếu muốn lấy địa chỉ liên lạc thứ 5 trong Con- tacts Content Provider thì URI sẽ là: content://contacts/people/5

Bảng 3.2: Các thành phần của chuỗi URI

Các bước để tạo một đối tượng ContentProvider bao gồm:

- Bước 1: Tạo ra một lớp con kế thừa lớp ContentProvider
- Bước 2: Xác định giá trị của chuỗi URI được sử dụng để truy cập đến nội dung
- Bước 3: Tạo cơ sở dữ liệu SQL, trong Android sử dụng SQLite
- Bước 4: Viết đè các phương thức truy vấn đến cơ sở dữ liệu
- Bước 5: Đăng ký sử dụng đối tượng Content Provider cho Activity trong file manifest sử dụng thẻ <provider>.

Các phương thức cần viết đè trong quá trình làm việc với đối tượng ContentProvider bao gồm:

- onCreate(): được gọi khi khởi động Content Provider
- query(): nhận một yêu cầu từ phía truy xuất và trả về một đối tượng con trỏ Cursor.
- insert(): thêm một bảng ghi mới vào content provider. delete(): xoá một bản ghi đang tồn tại trong content provider. update(): cập nhật một bản ghi đang tồn tại trong content
- provider.
- getType(): trả về kiểu MIME (Multipurpose Internet Mail Ex- tensions) của dữ liệu trong chuỗi URI được đưa ra.

Ví dụ sau đây sẽ giải thích chi tiết về các bước tạo một đối tượng Content Provider làm việc với cơ sở dữ liệu SQLite.

Ví dụ: Tạo cơ sở dữ liệu College gồm bảng students sau đó thiết kế giao diện như hình cho phép thêm các bảng ghi dữ liệu về sinh viên gồm trường Tên và Điểm bằng cách click vào nút Thêm sinh viên. Mỗi khi thêm một bản ghi thì hiển thị nội dung của chuỗi URI ra màn hình như hình (b). Khi người dùng click chọn nút Hiển thị danh sách sinh viên thì hiển thị thông tin từng sinh viên trong danh sách ra màn hình như hình c. Tạo một project mới có tên MyApplication và thực hiện các bước như sau:

a. Giao diện chính b. Khi click nút thêm sinh viên c. Khi click nút hiển thị danh sách sinh viên

Hình 6.5: Ví dụ về Content Provider

Bước 1: File MainActivity.java có nội dung như sau:

```
package ...
import android.app.Activity
import android.content.ContentValues
import android.database.Cursor
import android.net.Uri
import android.os.Bundle
import android.view.View
import android.widget.EditText
import android.widget.Toast
class MainActivity : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }

    // Phương thức cho phép thêm một bản ghi sinh viên vào cơ sở dữ liệu
       (CSDL)
    fun onClickAddName(view: View) {
        val values = ContentValues()

        // Lấy giá trị tên sinh viên
        values.put(
            StudentsProvider.NAME,
            findViewById<EditText>(R.id.editText2).text.toString()
        )

        // Lấy giá trị điểm sinh viên
        values.put(
            StudentsProvider.GRADE,
            findViewById<EditText>(R.id.editText3).text.toString()
        )

        val uri: Uri? =
```

```
            contentResolver.insert(StudentsProvider.CONTENT_URI, values)
            Toast.makeText(baseContext, uri.toString(),
            Toast.LENGTH_LONG).show()
        }

        // Phương thức lấy thông tin các sinh viên trong CSDL
        fun onClickRetrieveStudents(view: View) {
            val URL =
            "content://com.example.MyApplication.StudentsProvider"
            val students: Uri = Uri.parse(URL)
            val c: Cursor? = managedQuery(students, null, null, null,
            "name")

            // Di chuyển con trỏ đến bản ghi đầu tiên
            c?.let {
                if (it.moveToFirst()) {
                    do {
                        // Hiển thị ID, tên và điểm của sinh viên qua một
                       thông điệp Toast
                        Toast.makeText(
                        this,
                        it.getString(it.getColumnIndexOrThrow
                        (StudentsProvider._ID)) + ", " +
                         it.getString(it.getColumnIndexOrThrow
                        (StudentsProvider.NAME)) + ", " +
                         it.getString(it.getColumnIndexOrThrow
                         (StudentsProvider.GRADE)),
                            Toast.LENGTH_SHORT
                        ).show()
                    } while (it.moveToNext())
                }
            }
        }
}
```

Bước 2: Tạo file StudentsProvider.java và soạn thảo nội dung như sau:

```
package ...
import android.content.*
import android.database.Cursor
import android.database.SQLException
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper
import android.net.Uri
import android.text.TextUtils

class StudentsProvider : ContentProvider() {
    companion object {
        const val PROVIDER_NAME =
```

```
    "com.example.MyApplication.StudentsProvider"
    const val URL = "content://$PROVIDER_NAME/students"
    val CONTENT_URI: Uri = Uri.parse(URL)

    // Các hằng số tương ứng với tên cột
    const val _ID = "_id"
    const val NAME = "name"
    const val GRADE = "grade"

    // Định danh URI
    const val STUDENTS = 1
    const val STUDENT_ID = 2

    // Matcher URI
    val uriMatcher = UriMatcher(UriMatcher.NO_MATCH).apply {
        addURI(PROVIDER_NAME, "students", STUDENTS)
        addURI(PROVIDER_NAME, "students/#", STUDENT_ID)
    }

    // Cấu trúc CSDL
    const val DATABASE_NAME = "College"
    const val STUDENTS_TABLE_NAME = "students"
    const val DATABASE_VERSION = 1
    const val CREATE_DB_TABLE = """
        CREATE TABLE $STUDENTS_TABLE_NAME (
            $_ID INTEGER PRIMARY KEY AUTOINCREMENT,
            $NAME TEXT NOT NULL,
            $GRADE FLOAT NOT NULL
        );
    """
}

private lateinit var db: SQLiteDatabase
// Lớp trợ giúp SQLiteOpenHelper
private class DatabaseHelper(context: Context) :
    SQLiteOpenHelper(context, DATABASE_NAME, null,
    DATABASE_VERSION) {
    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL(CREATE_DB_TABLE)
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int,
    newVersion: Int) {
        db.execSQL("DROP TABLE IF EXISTS $STUDENTS_TABLE_NAME")
        onCreate(db)
    }
}
```

```
override fun onCreate(): Boolean {
    context?.let {
        val dbHelper = DatabaseHelper(it)
        db = dbHelper.writableDatabase
    }
    return this::db.isInitialized
}

override fun insert(uri: Uri, values: ContentValues?): Uri {
    val rowID = db.insert(STUDENTS_TABLE_NAME, null, values)
    if (rowID > 0) {
        val newUri = ContentUris.withAppendedId(CONTENT_URI, rowID)
        context?.contentResolver?.notifyChange(newUri, null)
        return newUri
    }
    throw SQLException("Failed to add a record into $uri")
}

override fun query(
    uri: Uri,
    projection: Array<out String>?,
    selection: String?,
    selectionArgs: Array<out String>?,
    sortOrder: String?
): Cursor {
    val qb = SQLiteQueryBuilder()
    qb.tables = STUDENTS_TABLE_NAME

    when (uriMatcher.match(uri)) {
        STUDENT_ID -> qb.appendWhere("$_ID =
         ${uri.pathSegments[1]}")
    }

    val orderBy = sortOrder ?: NAME
    val cursor = qb.query(db, projection, selection, selectionArgs,
    null, null, orderBy)
    cursor.setNotificationUri(context?.contentResolver, uri)
    return cursor
}

override fun delete(uri: Uri, selection: String?,
                    selectionArgs: Array<out String>?): Int {
    val count = when (uriMatcher.match(uri)) {
        STUDENTS -> db.delete(STUDENTS_TABLE_NAME, selection,
        selectionArgs)
        STUDENT_ID -> {
```

```
                val id = uri.pathSegments[1]
                db.delete(
                    STUDENTS_TABLE_NAME, "$_ID = $id" +
                            (if (!TextUtils.isEmpty(selection)) " AND
                               ($selection)" else ""), selectionArgs
                )
            }
            else -> throw IllegalArgumentException("Unknown URI $uri")
        }
        context?.contentResolver?.notifyChange(uri, null)
        return count
    }

    override fun getType(uri: Uri): String {
        return when (uriMatcher.match(uri)) {
            STUDENTS -> "vnd.android.cursor.dir/vnd.example.students"
            STUDENT_ID ->
            "vnd.android.cursor.item/vnd.example.students"
            else -> throw IllegalArgumentException("Unsupported URI:
            $uri")
        }
    }
}
```

Bước 3: Tạo file manifest có nội dung như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.MyApplication">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category
                   android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <provider
            android:name="StudentsProvider"

android:authorities="com.example.MyApplication.StudentsProvider"/>
```

```
    </application>

</manifest>
```

Bước 4: Tạo file activity_main.xml có nội dung như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="com.example.admin.exam_33.MainActivity">

    <TextView
        android:id="@+id/textView1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hướng dẫn sử dụng"
        android:textSize="30dp" />

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Content provider"
        android:textSize="30dp"
        android:layout_centerHorizontal="true" />

    <ImageButton
        android:id="@+id/imageButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@drawable/abc"
        android:layout_centerHorizontal="true" />

    <EditText
        android:id="@+id/editText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <EditText
        android:id="@+id/editText2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:hint="Tên" />

    <EditText
```

```
        android:id="@+id/editText3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:hint="Điểm" />

    <Button
        android:id="@+id/button2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Thêm sinh viên"
        android:onClick="onClickAddName" />

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hiện danh sinh viên"
        android:onClick="onClickRetrieveStudents" />

</RelativeLayout>
```

6.1.3. Các dịch vụ (Services)

Một dịch vụ (Service) là một thành phần được chạy bên trong hệ thống để xử lý các công việc trong thời gian dài. Ví dụ như dịch vụ nghe nhạc, tải dữ liệu từ mạng Internet vẫn được thực hiện bên trong hệ thống trong khi người dùng vẫn có thể tương tác với giao diện của các ứng dụng khác đang chạy.

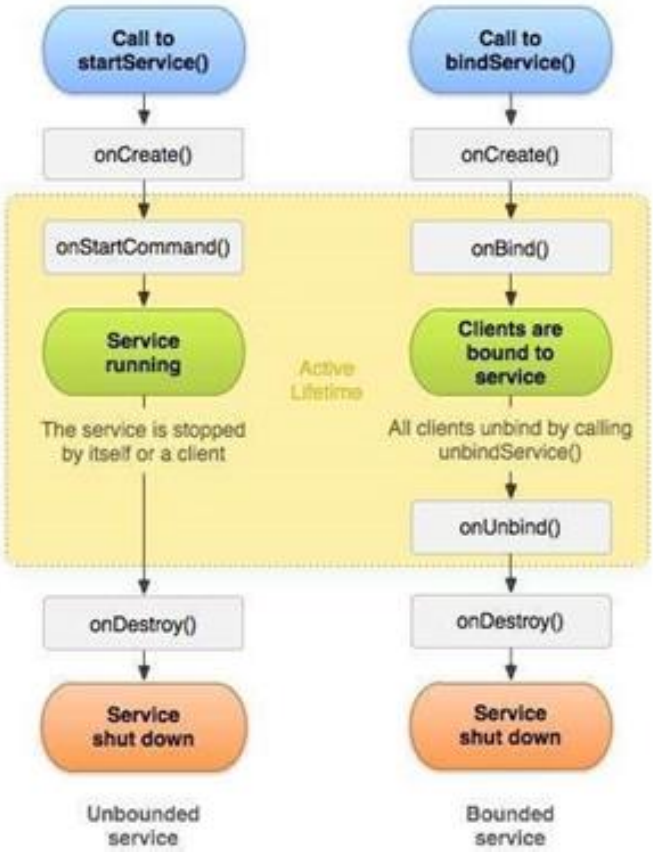
Sử dụng các dịch vụ có các lợi ích là: (1) khi một tiến trình (process) chạy sinh ra các tiểu trình (threads) và khi tiến trình kết thúc thì các tiểu trình của nó sẽ kết thúc, còn các dịch vụ có thể được hệ thống khởi động lại và kết thúc tuỳ theo mức độ quan trọng của nó; (2) các Activity và Broadcast Received độc lập với các tiểu trình mà nó sinh ra còn các Service thì gắn kết chặt chẽ với các tiểu trình nó sinh ra và có thể kết thúc các tiểu trình khi không cần thiết bằng cách sử dụng phương thức stopself().

Có hai loại dịch vụ:

- Started Service: một service thuộc loại started được triệu gọi khi một thành phần của ứng dụng khởi động nó bằng cách gọi phương thức startService(). Mỗi lần được khởi động, các dịch vụ chạy trên hệ thống là vô thời hạn.
- Bound Service: các Bound Services được gọi khi một thành phần ứng dụng liên kết với nó bằng cách gọi phương thức bindService(). Một Bound Service cung cấp một giao diện client-server cho phép các thành phần tương tác với Service gửi yêu cầu và nhận kết quả từ nó.

Mỗi Service có tập các phương thức cho phép người lập trình hiện thực hoá để giám sát sự thay đổi các trạng thái của nó và thực thi các công việc phù hợp ở mỗi trạng thái.

Sơ đồ mô tả vòng đời của các phương thức tương ứng với các trạng thái của từng loại Service từ khi được tạo cho tới khi bị huỷ.



Hình 6.6: Trình tự các trạng thái của Started Service và Bound Service

Ý nghĩa của từng phương thức như sau:

- onStartCommand(): phương thức này được gọi khi có một thành phần nào đó (các activities) đã yêu cầu khởi tạo Service bằng cách gọi phương thức startService(). Dịch vụ sẽ bị huỷ nếu người dùng.
- gọi phương thức stopSelf() hoặc stopService() khi không có nhu cầu sử dụng dịch vụ.
- onBind(): phương thức này được gọi khi có một thành phần khác gửi một liên kết tới Service thông qua việc gọi phương thức bindService(). Trong trường hợp này người lập trình phải cung cấp một giao diện để các client giao tiếp với Service và trả về một đối tượng IBinder.
- onUnbind(): phương thức này được gọi khi tất cả các client đã bị ngắt kết nối tới giao diện được tạo bởi các Service.
- onRebind(): phương thức này được gọi khi có một client mới kết nối tới Service mà trước đó tất cả các kết nối đã bị ngắt bởi phương thức onUnbind().

- onCreate(): phương thức này được gọi khi các Services được tạo lần đầu tiên bởi gọi phương thức onStartCommand() hoặc on- Bind().
- onDestroy(): phương thức này được gọi khi các Services đã không còn được sử dụng trong thời gian dài. Người lập trình nên gọi phương phức này để giải phóng các tài nguyên hệ thống.

Để sử dụng các Service cần khai báo trong file manifest như sau:

```
<manifest ... >
...
<application ... >
<service android:name=".ExampleService" />
...
</application>
</manifest>
```

Ví dụ: Tạo một lớp dịch vụ có tên HelloService liên tục in ra trong cửa sổ nhật lý Log dòng "Service running" 5 lần sau đó kết thúc dịch vụ. Lớp HelloActivity chứa hai nút nhấn "Start Service" và "Stop Service" cho phép khởi động và dừng Service HelloService như hình 0 – 45.

Hình 6.7: Giao diện lớp HelloActivity

File HelloService.java định nghĩa dịch vụ như sau:

```
import android.app.Service
import android.content.Intent
import android.os.IBinder
import android.util.Log

class HelloService : Service() {
    private val tag = "HelloService"
    private var isRunning = false
```

```
    override fun onCreate() {
        super.onCreate()
        Log.i(tag, "Service onCreate")
        isRunning = true
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId:
Int): Int {
        Log.i(tag, "Service onStartCommand")

        // Tạo một coroutine để chạy nền thay vì Thread
        Thread {
            repeat(5) {
                try {
                    Thread.sleep(1000)
                } catch (e: InterruptedException) {
                    e.printStackTrace()
                }
                if (isRunning) {
                    Log.i(tag, "Service running")
                }
            }
            stopSelf()
        }.start()

        return START_STICKY
    }

    override fun onBind(intent: Intent?): IBinder? {
        Log.i(tag, "Service onBind")
        return null
    }

    override fun onDestroy() {
        super.onDestroy()
        isRunning = false
        Log.i(tag, "Service onDestroy")
    }
}
```

File HelloActivity.java có nội dung như sau:

```
import android.app.Activity
import android.content.Intent
import android.os.Bundle
import android.view.View
import android.widget.Button
```

```
class HelloActivity : Activity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_hello)

        findViewById<Button>(R.id.start_service).setOnClickListener {
            val intent = Intent(this, HelloService::class.java)
            startService(intent)
        }

        findViewById<Button>(R.id.stop_Service).setOnClickListener {
            val intent = Intent(this, HelloService::class.java)
            stopService(intent)
        }
    }
}
```

File activity_hello.xml có nội dung như sau:

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context=".HelloActivity">

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentLeft="true"
        android:layout_alignParentStart="true"
        android:layout_below="@+id/textView"
        android:layout_marginTop="10dp"
        android:text="Ví dụ hướng dẫn sử dụng Service trong Android!!"
        android:textAppearance="?android:attr/textAppearanceMedium" />

    <Button
        android:id="@+id/start_service"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:layout_below="@+id/textView2"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="10dp"
```

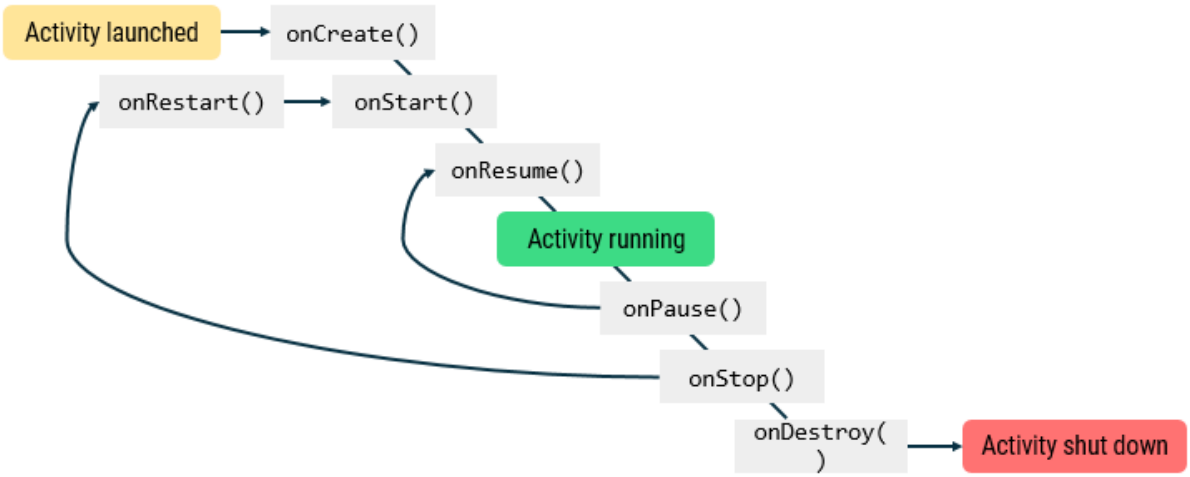
```
        android:text="Start Service" />

    <Button
        android:id="@+id/stop_Service"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:layout_below="@+id/start_service"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="10dp"
        android:text="Stop Service" />

</RelativeLayout>
```

File AndroidManifest.xml định nghĩa Service như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.javatechig.serviceexample">

    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme">

        <activity
            android:name=".HelloActivity"
            android:label="@string/app_name">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <!-- Khai báo dịch vụ -->
        <service
            android:name=".HelloService"
            android:exported="false" />

    </application>

</manifest>
```

6.2. Vòng đời của Activity và Fragment

6.2.1. Vòng đời của Activity



Hình 6.8: Vòng đời của Activity

Trong Android, Activity đại diện cho một màn hình giao diện người dùng. Hệ điều hành quản lý vòng đời của Activity để tối ưu tài nguyên và đảm bảo trải nghiệm mượt mà. Một Activity có thể trải qua nhiều trạng thái khác nhau được kiểm soát bởi các phương thức vòng đời.

Các trạng thái trong vòng đời của Activity

Hình 6.9: Các trạng thái vòng đời của Activity

Bảng 6.3. Các hàm trong vòng đời Activity

Phương thức	Mô tả
`onCreate()`	Được gọi khi Activity được tạo lần đầu tiên. Đây là nơi khởi tạo giao diện, biến, và dữ liệu.

`onStart()`	Activity bắt đầu hiển thị lên màn hình nhưng chưa sẵn sàng để tương tác.
`onResume()`	Activity đã hiển thị hoàn toàn và sẵn sàng nhận tương tác từ người dùng.
`onPause()`	Activity bị che khuất một phần hoặc mất focus (ví dụ: hộp thoại xuất hiện trên màn hình).
`onStop()`	Activity bị ẩn hoàn toàn nhưng vẫn tồn tại trong bộ nhớ.
`onDestroy()`	Activity bị hệ thống hủy để giải phóng tài nguyên.
`onRestart()`	Được gọi khi Activity quay trở lại sau khi bị dừng.

Ví dụ về vòng đời của Activity

Giả sử chúng ta có một Activity hiển thị thông báo trong từng trạng thái vòng đời:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        Log.d("Lifecycle", "onCreate called")
    }

    override fun onStart() {
        super.onStart()
        Log.d("Lifecycle", "onStart called")
    }

    override fun onResume() {
        super.onResume()
        Log.d("Lifecycle", "onResume called")
    }

    override fun onPause() {
        super.onPause()
        Log.d("Lifecycle", "onPause called")
    }

    override fun onStop() {
        super.onStop()
        Log.d("Lifecycle", "onStop called")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d("Lifecycle", "onDestroy called")
    }
```

```
}
```

Tình huống thực tế

- Khi mở ứng dụng: onCreate() → onStart() → onResume()
- Khi nhấn nút Home: onPause() → onStop()
- Khi quay lại ứng dụng: onRestart() → onStart() → onResume()
- Khi xoay màn hình: onPause() → onStop() → onDestroy() → onCreate() → onStart() → onResume()
- Khi hệ thống đóng ứng dụng do thiếu bộ nhớ: onPause() → onStop() → onDestroy()

6.2.2. Vòng đời của Fragment

Fragment là một thành phần có thể tái sử dụng trong Android, thường được nhúng vào Activity. Nó giúp chia nhỏ giao diện để quản lý tốt hơn trên nhiều kích thước màn hình.

Bảng 6.4. Các trạng thái trong vòng đời của Fragment

Phương thức	Mô tả
`onAttach()`	Fragment được gắn vào Activity.
`onCreate()`	Khởi tạo Fragment nhưng chưa tạo giao diện.
`onCreateView()`	Tạo giao diện của Fragment (gắn layout XML).
`onViewCreated()`	Giao diện đã được tạo xong.
`onStart()`	Fragment bắt đầu hiển thị.
`onResume()`	Fragment sẵn sàng tương tác với người dùng.
`onPause()`	Fragment tạm dừng khi bị che khuất.
`onStop()`	Fragment không còn hiển thị nhưng vẫn giữ trong bộ nhớ.
`onDestroyView()`	Giải phóng tài nguyên giao diện.
`onDestroy()`	Fragment bị hủy hoàn toàn.
`onDetach()`	Fragment bị tách khỏi Activity chứa nó.

Ví dụ về vòng đời của Fragment

```
class MyFragment : Fragment() {
    override fun onAttach(context: Context) {
        super.onAttach(context)
        Log.d("Lifecycle", "onAttach called")
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

```
        Log.d("Lifecycle", "onCreate called")
    }

    override  fun  onCreateView(inflater:  LayoutInflater,  container:
ViewGroup?, savedInstanceState: Bundle?): View? {
        Log.d("Lifecycle", "onCreateView called")
        return  inflater.inflate(R.layout.fragment_layout,  container,
false)
    }

    override fun onStart() {
        super.onStart()
        Log.d("Lifecycle", "onStart called")
    }

    override fun onResume() {
        super.onResume()
        Log.d("Lifecycle", "onResume called")
    }

    override fun onPause() {
        super.onPause()
        Log.d("Lifecycle", "onPause called")
    }

    override fun onStop() {
        super.onStop()
        Log.d("Lifecycle", "onStop called")
    }

    override fun onDestroyView() {
        super.onDestroyView()
        Log.d("Lifecycle", "onDestroyView called")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d("Lifecycle", "onDestroy called")
    }

    override fun onDetach() {
        super.onDetach()
        Log.d("Lifecycle", "onDetach called")
    }
}
```

Sự khác biệt giữa Activity và Fragment

Fragment không thể tồn tại độc lập, nó cần một Activity chứa nó.

Khi Activity bị hủy, tất cả Fragment bên trong cũng bị hủy theo.

Khi thay thế Fragment (replace hoặc add), phương thức onDestroyView() có thể được gọi nhưng onDestroy() chưa chắc được gọi ngay.

Khi sử dụng Fragment, ta thường khai báo trong `Activity`:

```
supportFragmentManager.beginTransaction()
    .replace(R.id.fragment_container, MyFragment())
    .commit()
```

Khi muốn loại bỏ Fragment:

```
supportFragmentManager.beginTransaction()
    .remove(MyFragment())
    .commit()
```

6.2.3. Quản lý vòng đời hiệu quả trong Android

Sử dụng ViewModel để giữ dữ liệu: Khi xoay màn hình, Activity bị hủy và tạo lại, nên dữ liệu sẽ mất. ViewModel giúp lưu dữ liệu ngay cả khi Activity bị hủy.

```
class MyViewModel : ViewModel() {
    var counter = 0
}
```

Sử dụng onSaveInstanceState() để lưu trạng thái UI. Nếu cần lưu trạng thái giao diện khi ứng dụng bị tạm dừng, sử dụng:

```
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)
    outState.putString("KEY", "Hello World")
}
```

Giải phóng tài nguyên đúng cách:

- Trong Activity: Giải phóng trong onDestroy().
- Trong Fragment: Giải phóng trong onDestroyView().

Vòng đời của Activity và Fragment rất quan trọng trong việc quản lý tài nguyên, tối ưu bộ nhớ và đảm bảo ứng dụng chạy mượt mà. Hiểu rõ các trạng thái này giúp lập trình viên xử lý tốt hơn các tình huống như xoay màn hình, chạy nền, chuyển đổi giao diện mà không làm mất dữ liệu.

Bài tập bổ trợ

Bài tập 1: Giới thiệu các thành phần cơ bản

Yêu cầu:

- Tạo một ứng dụng sử dụng tất cả các thành phần cơ bản:
- Một Activity hiển thị giao diện.
- Một Service chạy nền để đếm thời gian.
- Một BroadcastReceiver lắng nghe sự kiện thay đổi kết nối mạng.
- Một ContentProvider để chia sẻ danh sách sinh viên giữa hai ứng dụng.

Bài tập 2: Vòng đời của Activity

Yêu cầu:

- Tạo một ứng dụng ghi log các trạng thái vòng đời của Activity (onCreate(), onStart(), onResume(),...).
- Thêm một TextView hiển thị trạng thái hiện tại của Activity.

```
override fun onResume() {
    super.onResume()
    textView.text = "Trạng thái: onResume"
}
```

Bài tập 3: Điều hướng giữa các màn hình

Yêu cầu:

- Tạo hai Activity và điều hướng giữa chúng bằng Intent.
- Thêm một IntentFilter để mở một Activity bằng một sự kiện từ bên ngoài.

Hướng dẫn:

Điều hướng với Intent:

```
val intent = Intent(this, SecondActivity::class.java)
startActivity(intent)
```

IntentFilter:

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
</intent-filter>
```

Bài tập 4: Navigation Component

Yêu cầu:

- Thiết kế một ứng dụng sử dụng Navigation Component để điều hướng giữa ba Fragment.
- Thêm một nút "Back" để quay lại màn hình trước.

Hướng dẫn:

Navigation Graph:

```
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/nav_graph"
    app:startDestination="@id/firstFragment">
    <fragment
        android:id="@+id/firstFragment"
        android:name="com.example.FirstFragment" />
    <fragment
        android:id="@+id/secondFragment"
        android:name="com.example.SecondFragment" />
</navigation>
```

Bài tập 5: Quản lý trạng thái giữa các màn hình

Yêu cầu:

- Tạo một ứng dụng có ba Fragment (danh sách sản phẩm, chi tiết sản phẩm, và giỏ hàng).
- Sử dụng ViewModel để quản lý trạng thái giỏ hàng giữa các Fragment.

Hướng dẫn:

```
ViewModel:class CartViewModel : ViewModel() {
    val cartItems = MutableLiveData<List<Product>>()
    fun addItem(product: Product) {
        val currentItems = cartItems.value ?: listOf()
        cartItems.value = currentItems + product
    }
}
```